

iOS App Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 4 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/mobile-app-development/ios-swift-training-in-pune>

Course Overview

iOS development is the highest-paid mobile-development specialisation in Pune — smaller hiring pool than Android (Android is ~95% of Indian smartphones; iOS is the urban / premium-segment minority), but compensation runs noticeably higher because the talent supply is thinner and the apps generally serve premium users. Pune iOS roles cluster at the high end of consumer-tech (Razorpay Pune for premium-merchant apps, Pine Labs Pune, Amagi connected-TV, BharatPe Pune), the captive R&D centres (Mercedes-Benz R&D India, BMW TechWorks for some apps), plus the IT services majors building iOS apps for global clients. Archer Infotech's iOS Development training in Pune teaches the platform as it is actually built in 2026 — Swift 6 with strict concurrency, SwiftUI as the default UI toolkit (UIKit is 'reading legacy code' for new apps), the Observation framework for state, async / await for concurrency, SwiftData for persistence, plus the production tail (App Store Connect, TestFlight, App Store Review Guidelines, Apple Developer Program). Classroom in Kothrud, online live, and weekend batches available.

Why Learn iOS in 2026

iOS development is the highest-paid mobile-development specialisation in Pune — Indeed Pune lists 280+ active iOS Developer / Mobile Developer (iOS) openings as of May 2026. Compensation runs roughly 15–25% above Native Android at equivalent experience because the talent supply is thinner. The biggest employers are Razorpay Pune (premium-merchant apps), Pine Labs Pune, Amagi (connected-TV apps), BharatPe Pune (some apps), Mercedes-Benz R&D India (CarPlay and connected-car apps), BMW TechWorks for some teams, plus the Pune-based iOS teams of major BFSI apps (HDFC, ICICI, Bajaj Finserv) and IT services majors (TCS, Infosys, Wipro, Cognizant) building iOS apps for global clients.

What changed in 2026: Swift 6 (Sept 2024) shipped strict concurrency by default — the compiler enforces data-race safety at compile time, eliminating the largest class of iOS production bugs. SwiftUI has fully eclipsed UIKit for new apps; UIKit is 'reading legacy code' territory in 2026 hiring. The Observation framework (Swift 5.9+) replaced ObservableObject for state; SwiftData replaced Core Data for persistence in new apps (with Core Data still common for legacy maintenance). async / await + Actors are the universal concurrency pattern. Xcode 16+ with the Swift Assist AI features have improved developer productivity.

What this means for hiring: 2026 Pune iOS JDs expect Swift 6 with strict concurrency, SwiftUI at depth, the Observation framework, async / await + Actors, SwiftData or Core Data, plus one published TestFlight / App Store app. Senior roles add UIKit interop, Combine for reactive flows where needed, plus iOS-specific patterns (StoreKit 2 for in-app purchase, App Intents for Siri / Shortcuts).

- 280+ active iOS Developer roles on Indeed Pune (May 2026)
- iOS compensation runs 15–25% above Native Android at equivalent experience
- Swift 6 with strict concurrency — data-race-free by default
- SwiftUI + Observation + async/await + SwiftData — the 2026 stack
- Strong Pune fintech / BFSI / consumer-tech / automotive (CarPlay) hiring

Who This Course Is For

This course fits you if

- Engineering, BCS, MCA, or BSc-CS student targeting iOS Developer roles (especially premium-pay segment)
- Working Android / Flutter / RN developer wanting to add iOS for cross-platform reach
- Working Swift developer on UIKit wanting to migrate to SwiftUI + Swift 6 strict concurrency
- Working developer in another language wanting to enter Apple-platform development
- Career restarter targeting Pune premium-segment mobile roles where iOS compensation is highest

This course is not the right fit if

- If you do not have a Mac (or access to one) — iOS development requires Xcode which runs only on macOS; we offer lab access but personal Mac is strongly recommended
- If you have no programming background — at least basic Java / Kotlin / Python / JavaScript / Swift is required
- If you cannot put in 8–10 hours per week of practice outside class
- If you only want a certificate sticker — Pune iOS hiring screens hard on real TestFlight / App Store apps
- If your goal is the largest Pune mobile hiring pool — that's Native Android
- If you have 4+ years of production SwiftUI experience — talk to us about advanced specialisations

Detailed Curriculum

Module 1: Swift 6 Language Foundations (Week 1)

Swift as a language, before any UI appears. Variables and constants, the type system and inference, optionals with `?`, `if let`, `guard let` and `nil-coalescing` — plus a clear account of why force-unwrapping with `!` is a crash waiting to happen.

Control flow, functions with argument labels, closures and trailing-closure syntax, and the collection types with `map`, `filter`, `reduce` and `compactMap`. Structs, classes and enums are introduced with associated values and the value-versus-reference distinction that shapes every later design decision.

- Variables, constants and type inference
- Optionals — `if let`, `guard let`, `nil-coalescing`
- Why force-unwrapping is a crash waiting to happen
- Control flow, `switch` and pattern matching
- Functions, argument labels and default values
- Closures and trailing-closure syntax
- Collections — `map`, `filter`, `reduce`, `compactMap`
- Structs, classes and enums with associated values
- Value versus reference semantics

Module 2: Protocols, Generics & Error Handling (Week 2)

The half of Swift that separates working code from idiomatic code. Protocols and protocol-oriented design, extensions and default implementations, associated types, and opaque return types with `some` and `any` — the distinction interviewers use to gauge real Swift depth.

Generics with constraints and `where` clauses, then error handling: `throws` and `try`, typed throws, `Result`, and `defer`.

Memory management closes the module — ARC, strong, weak and unowned references, and the retain cycles that closures create if capture lists are ignored.

- Protocols and protocol-oriented design
- Extensions and default implementations
- Associated types and protocol constraints
- `some` versus `any` — opaque and existential types
- Generics with constraints and `where` clauses
- `throws`, `try`, typed throws and `Result`
- `defer` and cleanup ordering
- ARC — strong, weak and unowned

- Retain cycles and closure capture lists

Module 3: Xcode 16, Swift Package Manager & Project Anatomy (Week 3)

The tooling every iOS job assumes you already know. Xcode 16 navigation, the build system, schemes, targets and configurations; simulator and physical-device runs with a free developer account; and the debugger — breakpoints, LLDB, the view hierarchy inspector and the memory graph.

Swift Package Manager covers dependency management, local packages and modularising an app into feature modules. The module also covers Info.plist and entitlements, asset catalogues, build settings and xcconfig files, and reading the compiler errors that Swift is notorious for phrasing unhelpfully.

- Xcode 16 navigation, schemes and targets
- Build configurations and xcconfig files
- Simulator and device runs with a free account
- Breakpoints, LLDB and the view hierarchy inspector
- The memory graph debugger
- Swift Package Manager dependencies
- Local packages and feature modularisation
- Info.plist, entitlements and capabilities
- Asset catalogues and app icons

Module 4: SwiftUI Foundations — Views, State & Layout (Week 4)

SwiftUI is the default toolkit for every new iOS app, and the course treats UIKit as interop rather than a starting point. The View protocol, view composition and why small views are the unit of reuse, and the modifier system where order genuinely changes the result.

State is the centre: @State for local value state, @Binding for two-way handoff, @Environment for ambient values, and the single-source-of-truth discipline. Layout covers VStack, HStack, ZStack, Grid, Spacer, alignment guides, and the SwiftUI layout negotiation of proposed size and reported size.

- The View protocol and view composition
- Modifiers and why order changes the result
- @State for local value state
- @Binding for two-way handoff
- @Environment and ambient values
- VStack, HStack, ZStack, Grid and Spacer
- Alignment guides and custom alignment
- Layout negotiation — proposed and reported size
- GeometryReader and when to avoid it

Module 5: Navigation, Lists & Forms in SwiftUI (Week 5)

Building multi-screen apps. UINavigationController with value-based destinations and NavigationPath for programmatic navigation — including the deep-link and restore-state cases that hand-rolled navigation gets wrong.

NavigationSplitView for iPad and Mac layouts, tab views, sheets, popovers and full-screen covers.

Lists cover ForEach with stable identity, sections, swipe actions, pull-to-refresh, searchable and lazy loading for large data. Forms and controls close the module — pickers, toggles, steppers, focus state, validation and keyboard handling.

- UINavigationController and value-based destinations
- NavigationPath for programmatic navigation
- NavigationSplitView for iPad and Mac
- Tab views, sheets, popovers, full-screen covers
- List, ForEach and stable identity
- Sections, swipe actions and pull-to-refresh
- searchable and lazy loading large data
- Form controls — pickers, toggles, steppers

- @FocusState, validation and keyboard handling

Module 6: State Management & App Architecture (Week 6)

State beyond a single screen. The Observation framework is the modern default — the @Observable macro replacing ObservableObject and @Published, with automatic dependency tracking that only re-renders views actually reading a changed property. ObservableObject is covered as legacy you will inherit.

Architecture then gets an honest treatment: the MV pattern SwiftUI's design implies, MVVM and where its view models genuinely help, and The Composable Architecture as the structured option larger teams adopt. Dependency injection through the environment closes the module.

- The @Observable macro and automatic tracking
- @State with observable reference types
- @Bindable for two-way binding to observables
- ObservableObject and @Published as legacy
- Single source of truth and lifting state
- MV versus MVVM in a SwiftUI codebase
- The Composable Architecture overview
- Dependency injection through the environment
- Modelling screen state as one value

Module 7: Swift Concurrency — async/await & Structured Tasks (Week 7)

Asynchronous Swift as it is written in 2026. async/await replacing completion handlers, structured concurrency with Task and async let, task groups for parallel work, and cancellation that propagates through the tree rather than being checked ad hoc.

AsyncSequence and AsyncStream cover streaming values, including bridging a delegate or notification API into a stream. The module also covers Task lifetime tied to a SwiftUI view with .task, priorities, and bridging legacy completion-handler APIs with continuations — the migration every real codebase needs.

- async / await replacing completion handlers
- Task, async let and structured concurrency
- Task groups for parallel work
- Cancellation and cooperative cancellation checks
- AsyncSequence and AsyncStream
- Bridging delegates and notifications into streams
- The .task modifier and view-tied lifetime
- Task priorities and detached tasks
- Continuations for legacy completion handlers

Module 8: Actors, Sendable & Swift 6 Strict Concurrency (Week 8)

The largest change Swift has made in years, and a reliable interview subject. Actors for data-race-free shared mutable state, actor isolation and the await that crossing it requires, and reentrancy — the subtlety that surprises people who treat an actor as a lock.

@MainActor for UI isolation, global actors, and Sendable conformance with the checking that Swift 6 language mode makes an error rather than a warning. The module works through migrating a real codebase to strict concurrency, including nonisolated, @unchecked Sendable and when each is legitimate.

- Actors and data-race-free shared state
- Actor isolation and crossing it with await
- Actor reentrancy and its surprises
- @MainActor and UI isolation
- Global actors and custom isolation domains
- Sendable conformance and checking
- Swift 6 language mode — warnings become errors

- nonisolated and @unchecked Sendable
- Migrating a codebase to strict concurrency

Module 9: Networking — URLSession, Codable & Error Handling (Week 9)

Talking to a backend the Apple-native way. URLSession with async/await, request construction, headers and query items, and the response and status-code handling that separates a real client from a demo.

Codable covers decoding and encoding, coding keys, nested and heterogeneous JSON, date strategies and custom decoding for the APIs that do not match your model. Errors are modelled as a typed enum spanning transport, HTTP and decoding failures. Authentication, token refresh, retry, caching, background downloads and mocking with URLProtocol close the module.

- URLSession with async / await
- Request construction, headers and query items
- Status-code and response handling
- Codable, coding keys and nested JSON
- Date strategies and custom decoding
- Typed error enums for transport, HTTP and decoding
- Authentication, token refresh and retry
- URLCache and background downloads
- Mocking the network with URLProtocol

Module 10: Persistence — SwiftData, Core Data & Keychain (Week 10)

Data that survives relaunch. SwiftData as the modern default — @Model, the model container and context, @Query in SwiftUI, relationships, delete rules, predicates, sorting and schema migration. Core Data is covered as legacy, because a large share of Pune iOS work involves inheriting it, along with the SwiftData interoperability path.

UserDefaults handles preferences, the Keychain handles tokens and credentials, and the file system handles documents and caches. Offline-first design closes the module: local store as source of truth, sync, conflict resolution and CloudKit for cross-device.

- SwiftData @Model, container and context
- @Query, predicates, sorting and relationships
- Delete rules and cascading behaviour
- Schema migration in SwiftData
- Core Data as inherited legacy
- UserDefaults for preferences
- Keychain for tokens and credentials
- File system — documents, caches, app groups
- Offline-first, sync and CloudKit

Module 11: Animations, Gestures & Advanced SwiftUI (Week 11)

The polish that makes an app feel like it belongs on the platform. Implicit animation with the .animation modifier, explicit with Animation, transitions, phase and keyframe animators, and matchedGeometryEffect for hero-style movement between screens.

Gestures cover tap, drag, magnify and rotate, plus composition and simultaneous recognition. Advanced SwiftUI closes the module: Canvas for custom drawing, custom Layout conformance, scroll transitions and scrollTargetBehavior, visionOS and widget awareness, and UIViewRepresentable for the UIKit interop every real app eventually needs.

- Implicit and explicit animation
- Transitions, phase and keyframe animators
- matchedGeometryEffect for hero transitions
- Gestures — tap, drag, magnify, rotate
- Gesture composition and simultaneous recognition
- Canvas for custom drawing

- Custom Layout conformance
- Scroll transitions and scrollTargetBehavior
- UIViewRepresentable and UIKit interop

Module 12: Apple Platform Features & System Integration (Week 12)

The capabilities that only exist because the app is on Apple's platform. Push notifications through APNs and the UserNotifications framework — permissions, categories, actions, rich content and notification service extensions.

StoreKit 2 covers in-app purchase and subscriptions with the modern async API and transaction verification. App Intents expose features to Siri, Shortcuts and Spotlight; WidgetKit adds home-screen and Lock Screen widgets; and Sign in with Apple, LocalAuthentication for Face ID and Touch ID, Universal Links, and the App Tracking Transparency and privacy-manifest requirements close the module.

- APNs and the UserNotifications framework
- Notification categories, actions and rich content
- StoreKit 2 purchases and subscriptions
- Transaction verification and entitlement checks
- App Intents for Siri, Shortcuts and Spotlight
- WidgetKit home-screen and Lock Screen widgets
- Sign in with Apple
- Face ID and Touch ID with LocalAuthentication
- Universal Links, ATT and privacy manifests

Module 13: Testing with Swift Testing & XCTest (Week 13)

Tests that hold up as an app grows. Swift Testing is taught as the default for new code — `@Test`, `#expect` and `#require`, parameterised tests, traits and tags, and its async and parallel execution model. XCTest is covered as the framework you will still meet in existing projects.

Test design gets real attention: dependency injection for testability, fakes and protocol-based test doubles, testing async code and actors, and testing SwiftUI view models. XCTest covers UI automation, page-object structure, launch arguments for deterministic state, and reducing flakiness.

- Swift Testing — `@Test`, `#expect` and `#require`
- Parameterised tests, traits and tags
- Async and parallel test execution
- XCTest as the framework in existing projects
- Dependency injection for testability
- Protocol-based fakes and test doubles
- Testing async code and actors
- XCTest, page objects and launch arguments
- Reducing UI-test flakiness

Module 14: Performance, Instruments & App Size (Week 13)

Measuring before optimising, using the tools Apple ships. Instruments for time profiling, allocations, leaks and the SwiftUI instrument that shows which views re-render and why — the fastest route to fixing a sluggish screen.

Launch-time optimisation, main-thread blocking, list and image performance, and the SwiftUI-specific traps around identity and unnecessary body evaluation. Memory work covers leaks, retain cycles and the memory graph. App size closes the module — asset optimisation, on-demand resources, App Thinning and bitcode-era build settings.

- Instruments — time profiler and allocations
- The SwiftUI instrument and view re-render causes
- Launch-time optimisation and pre-main cost
- Main-thread blocking and hitches
- List and image performance
- SwiftUI identity and unnecessary body evaluation

- Leaks, retain cycles and the memory graph
- Asset optimisation and on-demand resources
- App Thinning and download size

Module 15: Release Engineering — Signing, TestFlight & App Store (Week 14)

The part that blocks more first-time iOS developers than any language feature. Certificates, identifiers, provisioning profiles and the signing model explained rather than memorised, plus automatic versus manual signing and resolving the failures each produces.

App Store Connect covers app records, build upload with Xcode Cloud or `xcodebuild` and `alttool`, TestFlight internal and external testing with beta review, and the App Store submission flow — screenshots, metadata, privacy nutrition labels, age rating and export compliance. Common rejection reasons and phased release close the module.

- Certificates, identifiers and provisioning profiles
- Automatic versus manual signing
- Resolving common signing failures
- App Store Connect app records and versioning
- Build upload with Xcode Cloud and `xcodebuild`
- TestFlight internal and external testing
- Screenshots, metadata and privacy nutrition labels
- Common App Store rejection reasons
- Phased release and staged rollout

Module 16: Capstone Project & Placement Preparation (Weeks 15–16 + 1 week placement prep)

Full-time capstone work followed by structured interview preparation. You build and ship a complete iOS application — SwiftUI throughout, Observation-based state, structured concurrency, a `URLSession` data layer with `SwiftData` offline storage, push notifications, an App Intent or widget, tests, and a TestFlight build real people can install.

Interview preparation matches Pune iOS panels: a Swift language round covering optionals, value semantics and ARC, a concurrency round on actors and `Sendable`, a SwiftUI round on state and identity, and a mobile system-design round. Resume, LinkedIn and GitHub polish included.

- Capstone implementation, release and README
- Shipping a TestFlight build to real testers
- Swift language round — optionals, ARC, value semantics
- Concurrency round — actors, `Sendable`, isolation
- SwiftUI round — state, identity, re-render behaviour
- Mobile system design — offline, sync, pagination
- Resume and LinkedIn rewrite for iOS JDs
- GitHub portfolio polish
- HR mock interview and salary negotiation

Module 17: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI

- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

Production iOS App with SwiftUI + Observation + SwiftData

A complete production-grade iOS app — pick a real domain (expense tracker, recipe finder, fitness tracker, BFSI-style finance dashboard, news reader). SwiftUI 100% (no UIKit ceremony), @Observable macro for state, async / await for concurrency with strict Swift 6 conformance, SwiftData for persistence, URLSession for a real public API, push notifications via APNs, biometric auth, plus Swift Testing tests. Published to TestFlight (Apple Developer Program ~Rs. 8,500 / year, paid by student). Outcome: a public GitHub repository plus TestFlight beta link.

- Technology: Swift 6 + SwiftUI
- Technology: Observation framework
- Technology: SwiftData
- Technology: async / await + Actors
- Technology: URLSession + Codable
- Technology: APNs push notifications
- Technology: Swift Testing
- Technology: TestFlight beta

Real-Time / Live-Data App

A real-time iOS app — chat / collaboration / live-feed app. WebSocket via URLSessionWebSocketTask, optimistic UI, push notifications for offline message delivery. Demonstrates the patterns Pune fintech / consumer-app teams hire on for real-time iOS work.

- Technology: Swift 6 + SwiftUI
- Technology: URLSessionWebSocketTask
- Technology: Async streams
- Technology: APNs notifications
- Technology: Offline message queue

Apple-Platform-Integrated Showcase App

An app that meaningfully integrates with Apple platform features — Sign in with Apple, biometric auth, Siri / Shortcuts via App Intents, in-app purchase via StoreKit 2, Universal Links, optionally Widgets / Live Activities. Demonstrates the depth Pune premium-segment iOS roles test for.

- Technology: Swift 6 + SwiftUI
- Technology: Sign in with Apple
- Technology: App Intents (Siri / Shortcuts)
- Technology: StoreKit 2
- Technology: Universal Links
- Technology: Widgets / Live Activities (optional)

Career Outcomes

iOS Developer is the highest-paid mobile-development specialisation in Pune in 2026 — Indeed Pune lists 280+ active openings, with compensation roughly 15–25% above Native Android at equivalent experience because the talent supply is thinner. The biggest Pune employers are Razorpay Pune (premium-merchant apps), Pine Labs Pune, Amagi (connected-TV apps), BharatPe Pune, Mercedes-Benz R&D India (CarPlay / connected-car), plus the Pune iOS teams of major BFSI apps and IT services majors.

What pulls an iOS developer above the median band: SwiftUI fluency with Swift 6 strict concurrency, demonstrable Observation framework + async / await depth, one published TestFlight or App Store app, plus testing discipline (Swift Testing + XCTest). Apple-platform integration (Sign in with Apple, App Intents, StoreKit 2) signals senior-iOS thinking on Pune premium-segment hiring panels.

Senior iOS Developer bands at the top end are reported as national figures (Pune-specific Indeed pages do not exist for these specific titles); Pune trends within $\pm 10\%$ of these figures.

Roles after this course

- iOS Developer
- Mobile Developer (iOS)
- Apple Platform Developer (iOS + macOS extension)
- Junior Mobile Engineer
- Software Engineer (iOS / Swift)

Indicative salary bands

- iOS Developer (Pune) - Rs. 7,75,000 per year average (source: Indeed Pune (iOS Developer))
- Junior iOS Developer (Pune entry, <2 years) - Rs. 4,00,000 – Rs. 7,50,000 per year (source: AmbitionBox Pune iOS Developer)
- Mid-level iOS Developer (Pune, 3–5 years) - Rs. 12,00,000 – Rs. 20,00,000 per year (source: Glassdoor Pune iOS Developer)
- Senior iOS Developer (Pune, 5–8 years) - Rs. 19,00,000 – Rs. 32,00,000 per year (source: Glassdoor Pune Senior iOS Developer)
- Lead iOS Engineer (national, 8+ years) - Rs. 30,00,000 – Rs. 52,00,000 per year (source: 6figr India Lead iOS Engineer (Pune $\pm 10\%$))

Companies hiring for these skills

- Razorpay (Pune, premium-merchant apps)
- Pine Labs (Pune)
- Amagi (connected-TV)
- BharatPe (Pune)
- Mercedes-Benz R&D India (CarPlay / connected-car)
- BMW TechWorks India
- HDFC Bank (Pune mobile teams)
- ICICI Bank (Pune mobile)
- Bajaj Finserv
- Kotak Mahindra Bank
- TCS
- Infosys
- Wipro
- Cognizant
- Capgemini
- Atos / Eviden

Batch Modes and Duration

Duration: 12 weeks of structured curriculum plus 1 week of capstone and interview preparation (~3.5 months total). The original 4-month listing reflects optional extended evening format.

Classroom

- Location: Archer Infotech, Kothrud, Pune (lab Macs available for students without personal Mac)
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- Personal Mac strongly recommended (or lab Mac access for in-person students)
- GitHub for code reviews
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~6 months instead of 3.5.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession. Apple Developer Program fee (~Rs. 8,500 / year — required for TestFlight / App Store publishing) is paid by the student.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2–3 instalments at no extra cost
- Corporate sponsorship — invoiced with GST

Placement Support

Placement support starts from week 9. By the end of the curriculum your resume highlights real published TestFlight / App Store iOS apps, your GitHub has at least two production-style repositories, and you have completed at least three mock technical interviews against question banks from Pune iOS hiring teams.

We say placement support, not placement guarantee. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Week 9 — resume + LinkedIn rewrite for iOS JDs
- Week 10 — GitHub portfolio cleanup, TestFlight beta links
- Weeks 11–12 — three rounds of mock technical interviews
- Week 13 — HR mock and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network at 12+ partner companies (with extra emphasis on Pune fintech / BFSI / consumer-tech)
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- Razorpay (Pune)
- Pine Labs (Pune)
- Amagi
- BharatPe (Pune)
- Mercedes-Benz R&D India
- Bajaj Finserv
- Kotak Mahindra Bank
- TCS
- Infosys
- Cognizant
- Capgemini

Prerequisites and How to Start

Prerequisites: at least basic programming background (any language), basic OOP understanding, willingness to commit 8–10 hours per week of practice, plus access to a Mac (personal Mac strongly recommended; we offer lab access for in-person students who don't yet have one).

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment — Apple ID + Apple Developer Program signup recommended (paid before publishing)
- Show up to day one with a Mac (personal or lab access) running macOS 14+ with Xcode 16+ pre-installed

Frequently Asked Questions

Do I need a Mac for iOS development?

Yes — Xcode runs only on macOS. Personal Mac is strongly recommended. We offer lab access for in-person students who don't yet have one, but for sustained learning a personal Mac is necessary.

How long does iOS training in Pune take at Archer Infotech?

Approximately 3.5 months — 12 weeks of structured curriculum plus 1 week of capstone. Original 4-month listing is optional extended evening format. Weekend batch ~6 months.

What is the salary of an iOS Developer in Pune?

Indeed Pune Rs. 7.75 lakh average — roughly 15–25% above Native Android at equivalent experience. Junior Rs. 4–7.5 lakh per AmbitionBox. Mid-level Rs. 12–20 lakh per Glassdoor. Senior Rs. 19–32 lakh. Lead Rs. 30–52 lakh nationally with Pune $\pm 10\%$.

SwiftUI or UIKit?

SwiftUI as the default for new apps. UIKit covered as 'reading legacy code' only. New Pune iOS work in 2026 is overwhelmingly SwiftUI.

Will I work on real projects?

Yes — three capstone projects with at least one published to TestFlight beta (Apple Developer Program fee paid by the student).

iOS, Flutter, or React Native?

Native iOS for the deepest Apple fluency and highest-paid mobile specialisation. Flutter or RN for cross-platform reach.

Are weekend iOS classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~6 months instead of 3.5.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000. Apple Developer Program fee (~Rs. 8,500 / year) paid by the student.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network (with extra emphasis on Pune fintech / BFSI / consumer-tech), resume / LinkedIn / GitHub rewrites, salary negotiation.

Is the named trainer actually teaching?

Amol Chougule personally leads every session of every batch.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.